

A PROJECT REPORT
HNSC-OSCC-MS Subtype Classifier

("HNSC-MS Subtype Classifier: Machine Learning
Classification of the Mesenchymal Molecular Subtype
of Head and Neck Squamous Cell Carcinoma from
Gene Expression Data")

A PROJECT REPORT

Submitted by

Aliza Kidwai

25MIB10020



VIT[®]

BHOPAL

GitHub Repository: <https://github.com/alizakidwai/oscc-ms-subtype-classifier.git>

Date: September 2026

1. Introduction

Head and Neck Squamous Cell Carcinoma (HNSCC), which includes Oral Squamous Cell Carcinoma (OSCC), is a common type of cancer found worldwide. However, HNSCC is not the same in every patient. According to large-scale genomic studies by The Cancer Genome Atlas (TCGA), HNSCC tumors can be divided into four different molecular subtypes based on their gene expression patterns: Basal, Mesenchymal, Atypical, and Classical (Walter et al., 2013; The Cancer Genome Atlas Network, 2015). Among these, the Mesenchymal (MS) subtype is particularly interesting because it is associated with an epithelial-to-mesenchymal transition (EMT) pattern, which is linked to more invasive tumor behavior.

In this project, a supervised machine learning method called a Random Forest classifier is used to predict whether a tumor sample belongs to the Mesenchymal subtype or not. The prediction is made using the gene expression profile of each tumor sample. Instead of using randomly generated or simulated data, this project uses real patient data from the publicly available TCGA-HNSC dataset. The data was obtained from the UCSC Xena Browser and the supplementary information provided with the original TCGA Nature 2015 publication.

The main reason for choosing this project is its connection to the topics covered in the Machine Learning Basics module. Concepts such as classification, feature importance, the bias-variance tradeoff, and the curse of dimensionality are all relevant to this problem. Gene expression data is especially useful for demonstrating these concepts because there can be thousands of genes (features) but only a relatively small number of patient samples. This makes the problem challenging and provides a good real-world example of how machine learning can be applied to biological data.

2. Problem Statement

Identifying the molecular subtype of a tumor can be difficult because it usually requires either analyzing the expression of a very large number of genes using computational methods or manually checking many marker genes. Both approaches can take time and may require specialist knowledge.

This project focuses on a simpler version of this problem.

The main question is:

Can a machine learning model use the expression values of a selected group of marker genes to classify an oral squamous cell carcinoma (OSCC) tumor as Mesenchymal (MS) or Other (Basal, Atypical, or Classical), while also providing information about which genes are important for the prediction?

2.1 Scope

- **Binary classification:** The model only predicts two classes: Mesenchymal (MS) and Other. The complete four-class classification of HNSCC is not included in this project and is considered a possible future improvement.
- **35-gene panel:** Instead of using the complete gene expression dataset containing around 20,000 genes, the project uses a selected panel of 35 genes based on existing HNSCC research. This makes the model easier to understand and also reduces the problem of having too many features compared to the number of patient samples.

- Real patient data: The project uses only real, publicly available patient data from the TCGA-HNSC dataset. No synthetic or simulated data is used for the final model.
- Command-line pipeline: The complete project is implemented as a command-line pipeline. It includes steps for loading the data, preprocessing it, training the model, evaluating its performance, and making predictions. A GUI or web application is not included because the main focus is on making the project simple, reproducible, and easy to run through scripts.
- Academic project only: This project is intended as a proof-of-concept for an academic setting. It is not designed for actual clinical diagnosis and does not include regulatory approval, clinical testing, or integration with hospital systems.

2.2 Target Users

- Primary users: For this project, the main user is the course evaluator, who can use the project to assess how well supervised machine learning concepts have been applied to a real biological dataset.
- Possible real-world users: In a real-world situation, bioinformatics researchers or laboratory technicians could use a similar system to get a quick first prediction of a tumor's molecular subtype from its gene expression data. The prediction could then help them decide which samples may need further analysis.

3. Functional Requirements

The system is built around five functional modules, each with a clearly defined input/output boundary, chained together into one command-line workflow (main.py).

ID	Module	Input	Output
FR-1	Data Ingestion (data_module.py)	Raw TCGA expression matrix (.gz) + subtype label file (.xlsx)	Merged, clean dataset (patients x genes)
FR-2	Preprocessing (preprocessing_module.py)	Merged dataset CSV	Schema-validated train/test splits
FR-3	Model Training (training_module.py)	Training split (X_train, y_train)	Trained Random Forest model + CV metrics
FR-4	Model Evaluation (evaluation_module.py)	Trained model + test split	Accuracy, precision, AUC, confusion matrix, feature importance
FR-5	Prediction (prediction_module.py)	New patient CSV (35 gene values)	Predicted survival probabilities for each gene

3.1 User Workflow

The project follows a simple step-by-step workflow through the command line. Each step performs a specific task, and the output from one step is used in the next

1. Build the dataset

Run `python main.py build-dataset` to combine the two raw data files and create the final dataset used for machine learning. This step only needs to be run once, or again if the original data is changed.

2. Train the model

Run `python main.py train` to train the Random Forest model. The training process uses 5-fold cross-validation to check how well the model performs on different parts of the training data. The trained model is then saved so that it can be used later.

3. Evaluate the model

Run `python main.py evaluate` to test the trained model on the separate test dataset. This step provides performance results and generates plots that help visualize how well the model is working.

4. Make a prediction

Run `python main.py predict --input <patient.csv>` to give the model gene expression data from a new patient. The model then predicts whether the sample is Mesenchymal (MS) or Other, along with information that helps explain the prediction.

Overall, this workflow provides more than the minimum three functional modules required for the project. It also gives a clear input and output structure and directly demonstrates the classification and prediction concepts covered in the machine learning course.

4.Non-Functional Requirements

Requirement	How it's addressed
Reliability	Every dataset and every prediction input is schema-validated (correct columns, no nulls, numeric types, known labels) before it reaches the model. Malformed input raises a clear, specific error instead of silently producing a wrong prediction.
Maintainability	All hyperparameters, file paths, and the gene panel live in one config.py. Each pipeline stage is an independent module with a single responsibility, so changing one stage (e.g. swapping the model type) does not require touching the others.
Usability	A single, consistent CLI (main.py) drives the whole pipeline with four simple subcommands. Predictions are returned as clean, human-readable JSON.
Logging / Monitoring	Every module logs to both the console and a persistent logs/app.log file (via a shared logger.py), giving an auditable record of what happened on each run -- dataset sizes, CV scores, metrics, errors.
Scalability	The gene panel, model hyperparameters, and even the underlying dataset can be swapped by editing config.py alone. Data ingestion is fully decoupled from training/evaluation/prediction, so a different cohort or a different classifier could be substituted without changing the downstream modules.

Resource Efficiency	Random Forest was deliberately chosen over a deep learning approach given the small sample size (279 patients) -- training completes in well under one second on a standard laptop CPU, with no GPU requirement.
----------------------------	---

5. System Architecture

The system follows a simple pipeline structure where each step is connected to the next. First, the raw data is collected and cleaned to create the dataset. This dataset is then used for preprocessing, model training, and evaluation.

After training, the Random Forest model is saved and reused by the prediction module to classify new patient samples. The project also uses `config.py` and `logger.py` across all modules to manage common settings and logging, which keeps the code organized and avoids repetition.

TCGA Nature 2015

Subtype labels, 279 patients

Build

Merg

Gene ex

279 patie

Pre

Valida

Trai

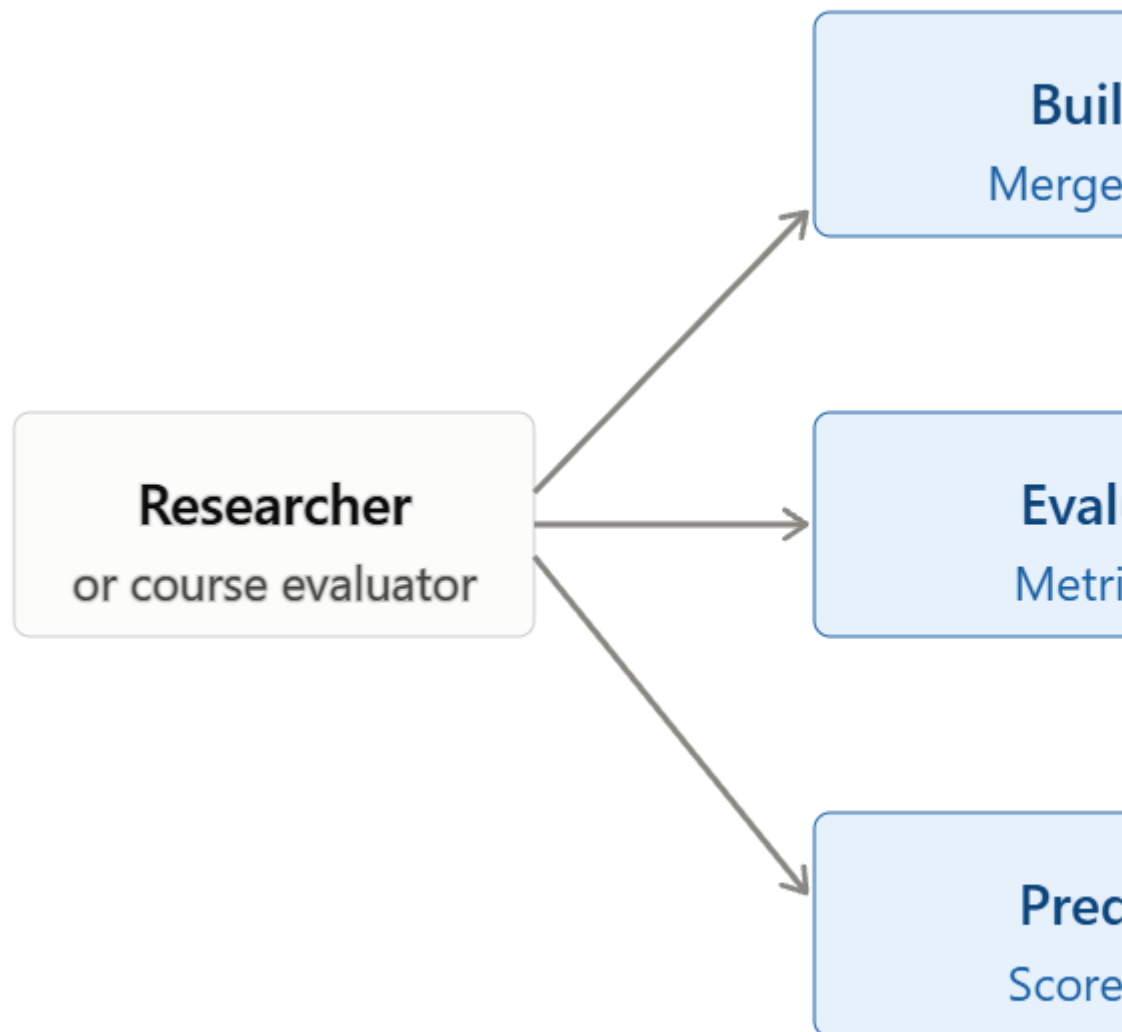
Random f

Train

6.Design Diagrams

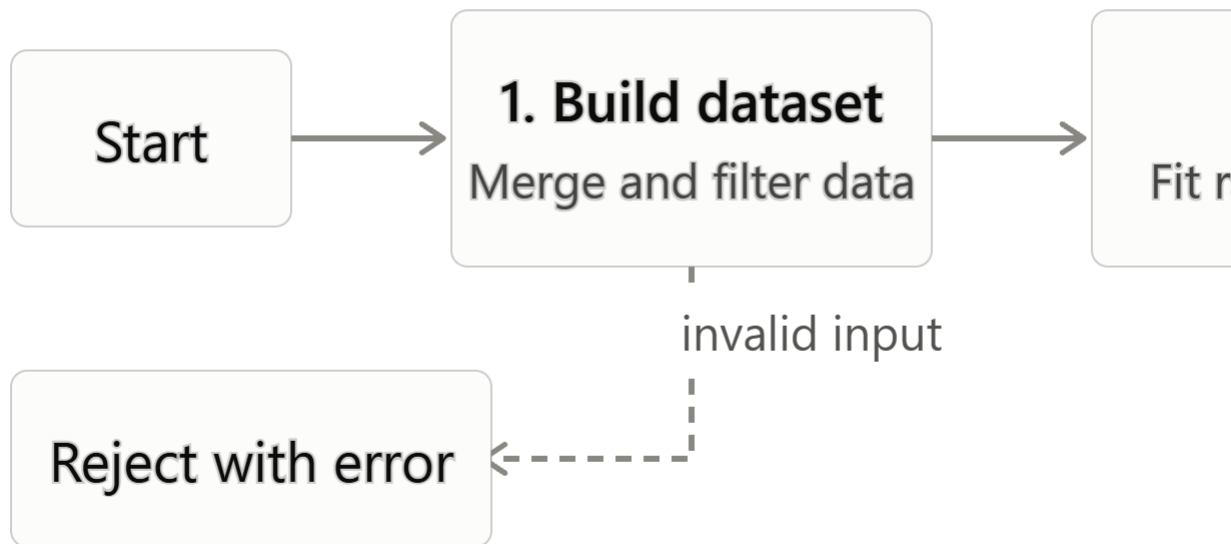
6.1 Use Case Diagram

The main user of the system is the researcher or evaluator who runs the pipeline through the command line. The main use cases are based on the different modules and CLI commands of the project. The “Validate Input Schema” use case is shared by both the dataset-building and prediction modules to make sure the input data is in the correct format.



6.2 Process Flow / Workflow Diagram

The workflow below traces a full run of the pipeline from start to finish across all four CLI stages, including the input-validation branch in the prediction stage.



6.3 Class / Component Diagram

Since this project is implemented as a set of focused, function-based Python modules rather than a deep class hierarchy, the diagram below is presented as a component diagram: each box represents one module, its public functions, and its dependencies on the shared config and logger components.

m
Command

Data module
Merges raw data

Training
Fits the model

P
Spl

Config
Shared by

6.4 Sequence Diagram

The sequence diagram below traces the most user-facing use case -- predicting a new patient's subtype -- from the CLI command down to the model and back.

User runs p

Load gene

Validate i

Load tra

Predict subty

Rank top co

Return r

6.5 Database / Storage Design

This project does not use a traditional database, so an ER diagram is not required. Instead, the project stores its data in files. The main dataset and prediction results are stored as CSV or JSON files, while the trained Random Forest model is saved using joblib. The structure of the main dataset file, `data/oscc_gene_expression.csv`, is described below similar to how a table structure would be shown in a database.

Column	Type	Description
sample_id	string	TCGA sample barcode, e.g. TCGA-CR-7383-01 (primary key / unique identifier)
<35 gene columns>	float	$\log_2(\text{RSEM}+1)$ normalized expression value for each gene in the literature panel
subtype_full	string	Original 4-class TCGA RNA subtype (Basal / Mesenchymal / Atypical / Classical) -- kept for reference
subtype	string	Binary modelling label derived from subtype_full: "MS" or "Other"

7. Design Decisions & Rationale

7.1 Why Random Forest

Random Forest was chosen because it works well with datasets that have many features and fewer samples, which is common in gene expression data. It also does not require feature scaling. Another important reason is that Random Forest provides feature importance, which helps us identify the genes that have the most influence on the prediction. Compared to a neural network, it is also less likely to overfit on a small dataset of 279 samples.

7.2 Why a Curated 35-Gene Panel Instead of the Full Transcriptome

The original dataset contains more than 20,000 genes but only 279 patient samples. Using all these genes could make the model overfit because there are far more features than samples. Therefore, we selected 35 genes based on existing HNSCC and EMT-related research. This makes the model simpler, easier to interpret, and more suitable for the available dataset.

7.3 Why Binary Classification

Instead of predicting all four HNSCC subtypes, this project focuses only on Mesenchymal (MS) vs Other. There are 75 MS samples and 204 Other samples, giving enough data for meaningful 5-fold cross-validation. This also keeps the project focused on the Mesenchymal subtype, which is the main subtype of interest in this study.

7.4 Why Stratified Cross-Validation and Balanced Class Weights

The dataset is imbalanced, with 75 MS samples compared to 204 Other samples. Stratified splitting helps maintain a similar class ratio in the training, testing, and cross-validation sets. We also use `class_weight="balanced"` so that the model gives more importance to the smaller MS class instead of mainly learning the majority class.

7.5 Why a CLI-First Design

The project uses a command-line interface because it makes the system simple and easy to run. Each stage can be executed using a separate command, making the pipeline easier to test and reproduce. It also avoids unnecessary GUI development and keeps the focus on the machine learning part of the project.

8. Implementation Details

8.1 Technology Stack

The project is mainly developed using Python. The main technologies used are:

- Python 3.10+ and argparse – for writing the program and handling CLI commands.
- pandas and NumPy – for loading, cleaning, and working with the data.
- scikit-learn – for the Random Forest model, cross-validation, data splitting, and evaluation metrics.
- Matplotlib and Seaborn – for creating graphs such as the confusion matrix, ROC curve, and feature-importance plots.
- openpyxl – for reading the Excel file containing the subtype labels.
- joblib – for saving and loading the trained model.
- pytest – for testing different parts of the project.

8.2 Data Ingestion Logic

The two raw files use different formats for patient IDs. The expression data uses IDs such as TCGA-CR-7383-01, while the subtype file uses IDs such as TCGA.BA.4074. The ingestion module converts both formats into a common patient ID so they can be matched.

Only primary tumor samples are kept, and the two files are then joined using the common ID. From the 566 samples in the expression data, 520 were primary tumor samples, and 279 had matching subtype information. These 279 samples were used for the final model.

8.3 Model Configuration

The Random Forest model is configured with 300 trees, balanced class weights, and a minimum leaf size of 2. The settings are stored in `config.py` instead of being directly written inside the training code. This makes it easier to change the model settings later.

8.4 Error Handling Strategy

The project uses separate error types for different problems, such as invalid data, incorrect input format, or prediction errors. These errors are handled by `main.py`, which displays a clear error message and stops the program properly. This is more useful than showing a long Python error or stack trace to the user.

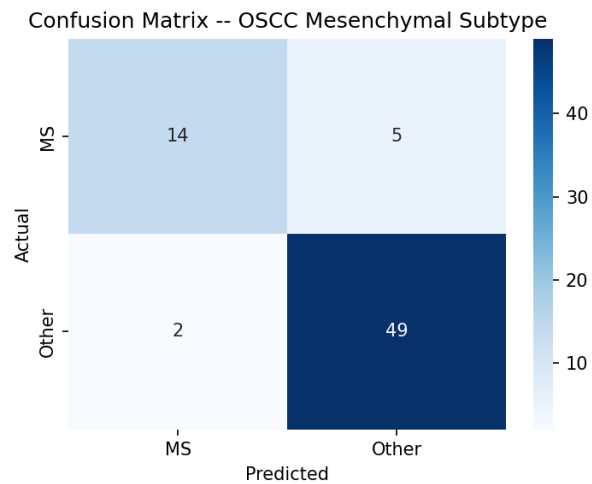
9.Screenshots / Results

9.1 Evaluation Metrics

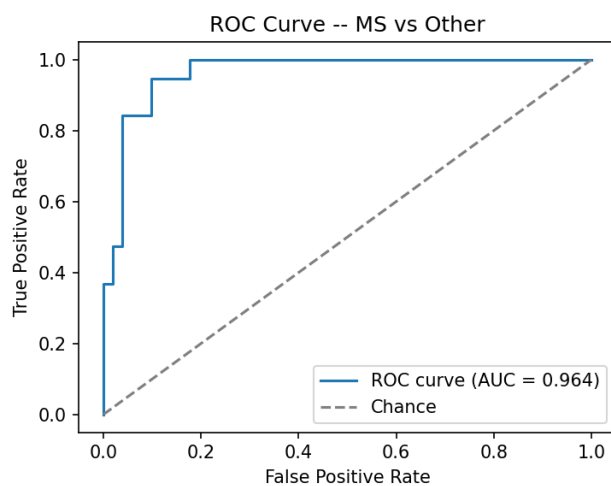
Metric	Value
5-fold CV ROC-AUC (mean)	0.957
Test set accuracy	0.92
Test set precision (MS)	0.88
Test set recall (MS)	0.84
Test set F1-score (MS)	0.86
Test set ROC-AUC	0.964

```
1 {  
2   "metrics": {  
3     "accuracy": 0.9285714285714286,  
4     "precision": 0.8888888888888888,  
5     "recall": 0.8421052631578947,  
6     "f1_score": 0.8648648648648649,  
7     "roc_auc": 0.9638802889576883  
8   },  
9 }
```

9.2 Confusion Matrix

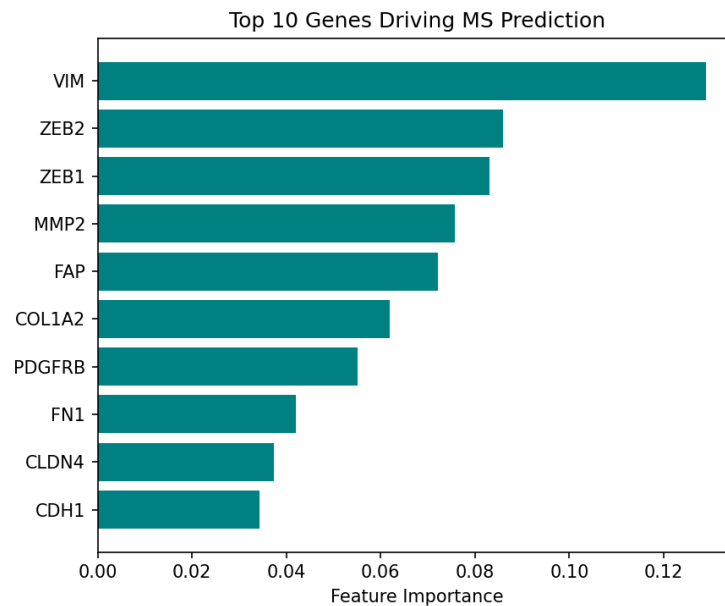


Confusion Matrix on the held-out test set (14/19 MS cases correctly identified).



ROC Curve, $AUC = 0.964$.

9.4 Feature Importance



Top 10 genes by Random Forest feature importance. VIM (vimentin), the textbook mesenchymal marker, ranks first, followed by the EMT transcription factors ZEB2 and ZEB1.

9.5 Example Prediction Output

Running python on a real held-out patient (true label: Mesenchymal) produces:

```
p genes: VIM, MMP2, COL1A2, ZEB2, ZEB1
[
  {
    "patient_id": "TCGA-CR-7383-01",
    "predicted_subtype": "MS",
    "probabilities": {
      "MS": 0.9699444444444445,
      "Other": 0.030055555555555558
    },
    "top_contributing_genes": [
      "VIM",
      "MMP2",
      "COL1A2",
      "ZEB2",
      "ZEB1"
    ]
  }
]
```

The model correctly predicted MS with 96.9% confidence for this patient, matching the ground-truth label.

10. Testing Approach

Test file	What it covers
test_data_module.py	Merged dataset shape, required columns present, no missing values, binary label correctly derived from the 4-class subtype, unique sample IDs
test_preprocessing_module.py	Schema validation passes on valid data and correctly raises on missing columns, null values, and unexpected labels; train/test split is stratified and non-overlapping
test_training_and_evaluation.py	Trained model only predicts known classes, beats a majority-class baseline, scores above-chance ROC-AUC, and all metrics fall within a valid [0, 1] range
test_prediction_module.py	Prediction output has the expected structure, predicted class is valid,

	probabilities sum to 1, top genes come from the known gene panel, and invalid input is correctly rejected
--	--

Beyond unit tests, the schema validation built into `preprocessing_module.py` and `prediction_module.py` acts as a form of continuous input-testing at runtime every dataset load and every prediction request is checked against the expected schema before it reaches the model, rather than being tested only once during development.

11. Challenges Faced

- Finding real patient data: Instead of using a ready-made OSCC dataset, we used real TCGA data. However, the required data was available from two different sources: the gene expression data from UCSC Xena and the subtype labels from the original 2015 Nature paper. These two sources had to be combined before the data could be used.
- Different patient ID formats: The two datasets used different formats for patient IDs. The expression data included extra information such as whether the sample was a tumor or normal tissue, while the subtype file used a simpler patient ID. Understanding these formats was necessary to correctly match the samples.
- Small number of samples: There were only 279 labelled patients compared to around 20,000 genes. Using all the genes could cause the model to overfit. Because of this, we used a smaller 35-gene panel based on previous research.
- Class imbalance: The dataset contains 75 MS samples and 204 Other samples. This imbalance could cause the model to mainly predict the larger class. To handle this, we used stratified splitting and balanced class weights.
- Avoiding data leakage: Another important challenge was making sure the test data was not used while training or tuning the model. We kept the cross-validation and final test evaluation separate so that the final performance result would be more reliable.

12. Learnings & Key Takeaways

- **Curse of dimensionality:** Working with real data made it clear why having too many features can be a problem. With 279 patients and around 20,000 genes, using all the genes could easily make the model overfit. This showed why selecting a smaller set of useful genes is important.
- **Feature importance:** Random Forest feature importance was useful for understanding what the model learned. The fact that VIM, a known marker associated with the mesenchymal phenotype, appeared as an important feature gave us more confidence in the model's results.
- **Class imbalance:** We learned that class imbalance can have a major effect on model performance. Stratified splitting helped ensure that both MS and Other samples were properly represented in each fold during cross-validation.
- **Data preparation:** A major part of the project was actually preparing and combining the real datasets. Handling different patient ID formats, identifying tumor samples, and merging the files took more effort than training the Random Forest itself.
- **Validation and error handling:** Adding input validation and clear error messages made the pipeline easier to test, debug, and understand. It also helped prevent incorrect data from being passed to the model.

13. Future Enhancements

- **Four-class classification:** Extend the current binary model to predict all four HNSCC subtypes: Basal, Mesenchymal, Atypical, and Classical.
- **External validation:** Test the model on an independent dataset, such as a GEO dataset, to check whether it works well on data other than TCGA.
- **Hyperparameter tuning:** Use methods such as Grid Search or Randomized Search to find better settings for the Random Forest model.
- **Model comparison:** Compare Random Forest with other machine learning models such as XGBoost and LightGBM.
- **Better explanations:** Use SHAP to provide more detailed and reliable explanations for why the model makes a particular prediction.
- **Clinical information:** Include additional patient information such as age, tumor stage, and HPV status along with gene expression data.
- **Web interface:** Develop a simple web interface for users who are not comfortable with the command line, while keeping the CLI version available for researchers and technical users.

14. References

1. The Cancer Genome Atlas Network. “Comprehensive genomic characterization of head and neck squamous cell carcinomas.” *Nature*, 517, 576–582 (2015).
<https://doi.org/10.1038/nature14129>
2. Walter, V., Yin, X., Wilkerson, M. D., Cabanski, C. R., Zhao, N., Du, Y., et al. “Molecular Subtypes in Head and Neck Cancer Exhibit Distinct Patterns of Chromosomal Gain and Loss of Canonical Cancer Genes.” *PLoS ONE*, 8(2), e56823 (2013).
<https://doi.org/10.1371/journal.pone.0056823>
3. UCSC Xena Browser. “TCGA Head and Neck Cancer (HNSC) Cohort – Gene Expression RNAseq.” Available at:
<https://xenabrowser.net/datapages/>
4. Breiman, L. “Random Forests.” *Machine Learning*, 45, 5–32 (2001).
5. Pedregosa, F., et al. “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, 12, 2825–2830 (2011).
Available at: <https://scikit-learn.org/>
6. McKinney, W. “Data Structures for Statistical Computing in Python.” *Proceedings of the 9th Python in Science Conference* (2010).